SentinelSSH

Analyseur de logs SSH — Sécurité défensive

Stage 1 Report

Portfolio Project — Team Formation, Brainstorming and MVP (Stage 1 - France)

Anil AKER
Holberton School Bordeaux
Projet de fin d'année — Demo Day / Titre RNCP niveau 5
github.com/anil-akr
30 septembre 2026

> **SentinelSSH en une phrase.** Une application web qui analyse les logs de connexion SSH d'un serveur Linux, repère les adresses IP suspectes, leur attribue un niveau de gravité et présente le résultat en tableau et en graphique, avec un historique des analyses.

Sommaire

1. Formation de l'équipe et définition des rôles 3

2. Brainstorming et évaluation des idées 5

3. Décision et affinage du concept 8

4. Résumé du MVP sélectionné 14

Annexe — Journal des décisions 14

Correspondance avec les consignes du Stage 1

Consigne Holberton	Où la trouver
Task 0 — Team Formation & roles definition	§1
Task 1 — Brainstorming and Idea evaluation	§2
Task 2 — Decision and Refinement	§3
Task 3 — Documentation (Summary)	§4
Livrable — Team Formation Overview	§1
Livrable — Ideas Explored (description et raisons du rejet)	§2.4 et §2.5
Livrable — Selected MVP Concept (résumé, raisons du choix, défis et opportunités)	§3.6, §3.10 et §4

1. Formation de l'équipe et définition des rôles

1.1 Composition de l'équipe et réunion de lancement

SentinelSSH est un projet mené en individuel. La réunion de lancement a donc pris la forme d'une **session de cadrage** : faire le point sur mon profil, mes points forts, mes axes de progrès et mes centres d'intérêt, pour choisir un projet réaliste et répartir le travail entre les différents rôles.

Membre	Parcours	Points forts	Axes de progrès	Centres d'intérêt
Anil AKER	1re année à Holberton School Bordeaux (école de développement)	Python, C, Shell · Linux (WSL) · bases réseau (TCP/UDP, SSH) · SQL	JavaScript et React · conception d'interfaces	Cybersécurité défensive · sécurité des infrastructures et des réseaux · GRC

Ce bilan a eu deux conséquences directes sur le projet : placer le cœur technique (l'analyse des logs) dans le langage que je maîtrise le mieux, Python, et prévoir dès le début du temps d'apprentissage pour React, ma principale zone de progrès.

1.2 Rôles attribués et justification

Rôle de coordination : j'assure le rôle de **chef de projet** (Project Manager) pendant tout le projet, et pas seulement pendant le Stage 1 : planification, suivi des tâches, respect des échéances, arbitrages.

Rôles techniques : le projet étant individuel, tous les rôles sont tenus par la même personne. Les nommer séparément reste utile : à chaque étape, je sais « avec quelle casquette » je travaille, je peux estimer le temps de chaque rôle et vérifier qu'aucune partie n'est oubliée.

Rôle	Responsabilités sur SentinelSSH	Pourquoi ce rôle, et pourquoi ainsi	Temps
Chef de projet	Planning, kanban GitHub, suivi des échéances, journal des décisions	Garder une vue d'ensemble et un rythme, même seul	10 %
Développeur back-end	Lecture des logs, règles de détection, API REST et connexion sécurisée en Python/Flask	C'est le cœur du projet, confié à mon point fort (Python)	25 %
Concepteur de la base de données	Schéma SQLite, requêtes, protection contre les injections SQL	Les résultats et l'historique doivent être stockés proprement ; le SQL est déjà pratiqué	5 %
Développeur front-end	Interface React responsive : 4 pages, composants, tableau, graphique	Rendre l'analyse lisible et démontrable ; compétence à acquérir, donc planifiée tôt	25 %
Ingénieur sécurité	Définir quoi détecter, sécurité de l'application, conformité RGPD	Cœur de ma spécialisation visée ; un outil de sécurité doit être sûr lui-même	10 %
Testeur (QA)	Tests automatiques, checklist manuelle, intégration continue	Un outil de sécurité qui se trompe est pire qu'inutile	10 %
Responsable déploiement	Mise en production locale (Gunicorn), intégration continue	Le déploiement documenté est une compétence du titre RNCP	5 %
Rédacteur technique	README, rapports, documentation de l'API et du déploiement	Nécessaire pour le jury et pour un futur utilisateur	10 %

QA (Quality Assurance) : le rôle qui vérifie que tout fonctionne comme prévu. Les pourcentages sont une estimation du temps total ; ils servent à planifier et seront réajustés pendant les revues hebdomadaires.

1.3 Normes de travail : communication, collaboration et décisions

Même en solo, je m'impose les règles d'une vraie équipe. Elles rendent le travail traçable pour l'équipe pédagogique et le jury.

Domaine	Outil	Norme appliquée
Suivi des tâches	GitHub Projects (kanban) + GitHub Issues	Une issue par tâche ou user story ; colonnes À faire / En cours / Terminé ; mise à jour à chaque session de travail
Code	Git + GitHub	Une branche par fonctionnalité, fusion par Pull Request, messages de commit clairs (feat:, fix:, test:, docs:) ; la branche main fonctionne toujours
Documentation	Google Docs (rapports) + README et dossier docs/ du dépôt	Toute l'information est centralisée et versionnée ; rien ne reste « dans la tête »
Communication	Messagerie de l'école (Slack/Discord) et e-mail	Point d'avancement hebdomadaire envoyé à l'équipe pédagogique ; questions posées dès qu'un blocage dépasse une demi-journée
Rythme	Revue hebdomadaire (en fin de semaine)	Bilan : ce qui est fait, ce qui reste, les blocages ; ajustement du planning
Décisions	Journal des décisions (docs/decisions.md)	Chaque décision importante est notée : date, contexte, options, choix et raison (voir l'annexe)

Règles de prise de décision

- Les choix se font sur des **critères écrits** (grille d'évaluation du §2.3), pas sur l'intuition seule.
- En cas de doute sur une fonctionnalité, **le MVP prime** : si elle menace l'échéance, elle passe en version suivante.
- Toute décision qui change le périmètre, la technologie ou le planning est inscrite au journal des décisions.

1.4 Parties prenantes

Partie prenante	Rôle	Attentes	Impact sur le projet
Anil AKER	Porteur du projet : développeur et chef de projet	Livrer un MVP fonctionnel, maîtrisé et bien documenté	**Élevé** : prend toutes les décisions
Utilisateur cible : administrateur système ou analyste sécurité junior	Bénéficiaire de l'outil	Repérer vite les intrusions SSH, sans outil lourd	**Élevé** : ses besoins définissent les fonctionnalités
Équipe pédagogique Holberton	Suivi et validation des étapes	Respect des consignes, des échéances et de la qualité	**Élevé** : valide chaque stage
Jury du demo day	Évaluation du projet	Une démonstration claire et un projet maîtrisé	**Élevé** : évaluation finale
Jury du titre RNCP niveau 5	Certification des compétences	Preuves concrètes des compétences du référentiel	**Élevé** : oriente la complétude technique (front, API, back, base)
Étudiants de la promotion	Testeurs et relecteurs volontaires	Des retours d'utilisation honnêtes	**Moyen** : améliorent l'ergonomie
Outils et données open source (fail2ban, dataset Loghub)	Références indirectes	—	**Faible** : point de comparaison et données de test

2. Brainstorming et évaluation des idées

2.1 Recherche : d'où viennent les idées

La recherche est partie de trois sources, filtrées par les contraintes du projet.

- **Problèmes réels :** tout serveur Linux accessible en SSH reçoit en continu des tentatives de connexion automatiques. Elles sont enregistrées dans les logs, mais ces fichiers deviennent vite trop longs pour être lus à la main.
- **Tendances du secteur :** la détection d'intrusion et l'analyse de logs sont au cœur du travail des équipes de sécurité (SOC, SIEM). Les exigences de conformité (RGPD, ISO 27001) poussent aussi à tracer et à justifier ce qui se passe sur un système.
- **Solutions existantes étudiées :** fail2ban (blocage automatique des IP), nmap (scan de ports), les SIEM open source comme Wazuh (surveillance complète, mais lourde à déployer).
- **Contraintes du projet :** 1 mois de développement, un seul développeur, une application complète avec front-end, API, back-end et base de données, une technologie JavaScript côté front, et les compétences du titre RNCP niveau 5 « Développeur web et web mobile » (RNCP 37674) à démontrer.

2.2 Session de brainstorming : 3 techniques

a) Questions « How Might We » (« Comment pourrions-nous… »)

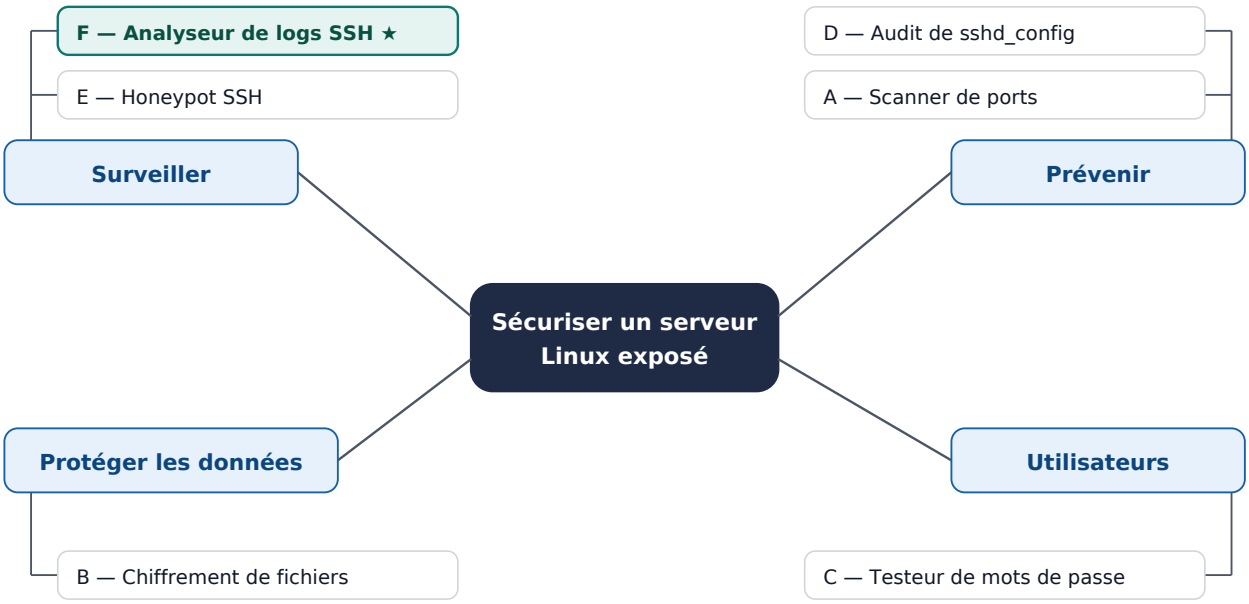
Cette technique transforme un problème en question ouverte, pour chercher des solutions sans se limiter trop tôt.

- Comment pourrions-nous aider un administrateur à repérer une attaque **sans lire ses logs ligne par ligne** ?
- Comment pourrions-nous rendre visible **le moment où un attaquant réussit à entrer** ?

- Comment pourrions-nous aider un administrateur junior à sécuriser son serveur **sans outil lourd à configurer** ?

b) Carte mentale (mind mapping)

À partir du thème central, j'ai exploré 4 directions. Chaque direction a produit une ou deux idées de projet (lettres A à F).



★ *idée retenue*

Carte mentale du brainstorming : 4 directions, 6 idées.

c) Méthode SCAMPER appliquée à l'existant

SCAMPER consiste à transformer une solution existante avec 7 questions. Je l'ai appliqué à fail2ban, la référence contre la force brute SSH. C'est ce travail qui a fait naître l'idée F.

Lettre	Question	Application à fail2ban → piste pour SentinelSSH
S — Substituer	Que peut-on remplacer ?	Remplacer le blocage automatique par une **décision humaine guidée** : l'outil propose la commande, l'admin décide
C — Combiner	Que peut-on associer ?	Associer analyse de logs, **visualisation web** et **historique** en base de données
A — Adapter	Que peut-on reprendre d'ailleurs ?	Adapter la vue d'ensemble d'un SIEM à l'échelle **d'un seul serveur et d'un admin junior**
M — Modifier	Que peut-on changer ou amplifier ?	Transformer une sortie texte en **tableau trié par gravité** et en graphique
P — Autre usage	À quoi d'autre cela peut-il servir ?	Analyser un incident **après coup**, ou servir d'outil pédagogique
E — Éliminer	Que peut-on supprimer ?	Supprimer la configuration complexe : un **simple envoi de fichier** suffit
R — Réorganiser	Que peut-on inverser ?	Au lieu de réagir à chaque échec, regarder l'ensemble et repérer **le succès qui suit une série d'échecs**

2.3 Critères d'évaluation

Chaque idée est notée de 1 (faible) à 5 (fort) sur 6 critères, soit un total sur 30. Les critères reprennent ceux proposés par la consigne (faisabilité, impact, alignement technique, évolutivité, risques), auxquels j'ai ajouté la complétude technique, exigée par le cadre du projet.

Critère	Ce qu'il mesure	Une note de 5 signifie…
Faisabilité	Réalisable seul, en 1 mois, avec mes compétences actuelles	Aucune technologie vraiment inconnue, périmètre maîtrisé
Impact	Résout un vrai besoin, apporte quelque chose par rapport à l'existant	Besoin fréquent et réel, valeur ajoutée claire
Alignement	Cohérence avec la cybersécurité défensive et la sécurité des infrastructures	Exactement dans ma spécialisation visée
Évolutivité	Capacité à grandir après le MVP (scalability)	Plusieurs versions futures évidentes
Risques	Risques techniques, légaux ou de délai	Risques faibles et faciles à maîtriser
Complétude	Mobilise front-end, API, back-end et base de données de façon naturelle	Les 4 couches ont une vraie raison d'exister

2.4 Idées explorées

Idée	Description	Décision et raison
A — Scanner de ports	Outil qui liste les ports ouverts d'une machine pour vérifier sa surface d'attaque.	**Rejetée.** nmap couvre déjà ce besoin avec une maturité impossible à égaler en 1 mois. Utilisé sur une machine sans autorisation, un scan peut être illégal. Peu de raisons d'avoir une base de données.
B — Chiffrement de fichiers	Programme en ligne de commande pour chiffrer et déchiffrer des fichiers.	**Rejetée.** Sujet de cryptographie, éloigné du réseau et de la surveillance. Coder sa propre cryptographie est une mauvaise pratique de sécurité. Pas de front ni de base naturels.
C — Testeur de mots de passe	Application web qui évalue la robustesse d'un mot de passe et donne des conseils.	**Rejetée.** Nombreux outils existants, fonctionnement simple, peu lié à la sécurité des infrastructures.
D — Audit de sshd_config	Outil qui lit la configuration du serveur SSH et signale les réglages dangereux (connexion root, mots de passe autorisés…).	**Rejetée pour le MVP, gardée pour la V2.** Très alignée (durcissement et conformité), mais le périmètre est étroit : un seul fichier, une liste de règles fixes, peu de données à stocker ou à visualiser.
E — Honeypot SSH	Faux serveur SSH qui attire les attaquants pour étudier leurs techniques.	**Rejetée pour le MVP, gardée pour plus tard.** Passionnante, mais elle demande un serveur exposé sur Internet, présente des risques réels pour l'infrastructure et dépasse largement 1 mois.
F — SentinelSSH	Application web qui analyse les logs SSH, détecte les attaques (force brute, compromission possible), les classe par gravité et garde un historique.	**Retenue.** Meilleur score (voir §2.5). Défensive, centrée sur l'infrastructure, complète techniquement et différente de fail2ban.

2.5 Classement des idées

Idée	Faisa-bilité	Impact	Aligne-ment	Évolu-tivité	Risques	Complé-tude	Total /30	Rang
F — SentinelSSH	**3**	**4**	**5**	**5**	**4**	**5**	**26**	**1er**
D — Audit de sshd_config	4	3	5	3	5	3	**23**	2e
C — Testeur de mots de passe	5	3	2	2	4	3	**19**	3e
E — Honeypot SSH	1	4	5	5	1	3	**19**	3e
A — Scanner de ports	5	2	3	2	3	2	**17**	5e
B — Chiffrement de fichiers	4	2	2	2	2	2	**14**	6e

Pour « Risques », une note de 5 signifie des risques faibles. C et E sont à égalité (19/30).

Ce que montre le classement

- **SentinelSSH arrive en tête (26/30)** alors que ce n'est pas l'idée la plus facile : sa faisabilité (3/5) est plus faible que celle du scanner de ports, car React est une technologie que je dois encore apprendre. Elle l'emporte grâce à son impact, son alignement et sa complétude.
- **L'audit de sshd_config (23/30) est un sérieux second.** Il a été écarté pour son périmètre trop étroit, mais il deviendra une fonctionnalité de SentinelSSH en V2 : les deux idées se complètent.
- **Le honeypot est la meilleure idée à long terme** (impact, alignement, évolutivité), mais la moins faisable et la plus risquée aujourd'hui.

3. Décision et affinage du concept

3.1 Comment la décision a été prise

1. **Notation** des 6 idées avec la grille du §2.3 : SentinelSSH obtient le meilleur score.
2. **Vérification de l'existant :** fail2ban traite déjà la force brute SSH. J'ai vérifié que SentinelSSH apporte une vraie différence (§3.6) avant de le retenir, pour ne pas refaire un outil existant.
3. **Confrontation aux contraintes :** le périmètre a été réduit jusqu'à tenir en 1 mois, seul, avec un front en React.
4. **Traçabilité :** la décision et ses raisons sont inscrites au journal des décisions (annexe, D1).

3.2 Le problème résolu

Un serveur Linux exposé en SSH subit en permanence des tentatives de connexion, dont beaucoup sont des **attaques par force brute** : un robot essaie des milliers de mots de passe à la suite. Ces tentatives sont écrites dans le fichier `auth.log`, mais le lire à la main est long et peu fiable. Le plus grave passe facilement inaperçu : **une IP qui a échoué des dizaines de fois, puis a fini par se connecter**.

3.3 La solution proposée

SentinelSSH analyse un fichier de logs SSH envoyé par l'administrateur. Il repère les IP suspectes grâce à 3 règles de détection, leur attribue un niveau de gravité (moyenne, élevée, critique), enregistre le résultat en base de données et l'affiche dans une application web : résumé chiffré, tableau trié par gravité, graphique et commande de blocage prête à copier. Un historique permet de retrouver les analyses passées et de repérer les IP qui reviennent. L'application est protégée par une connexion : les résultats ne sont visibles que par l'administrateur.

3.4 Public cible

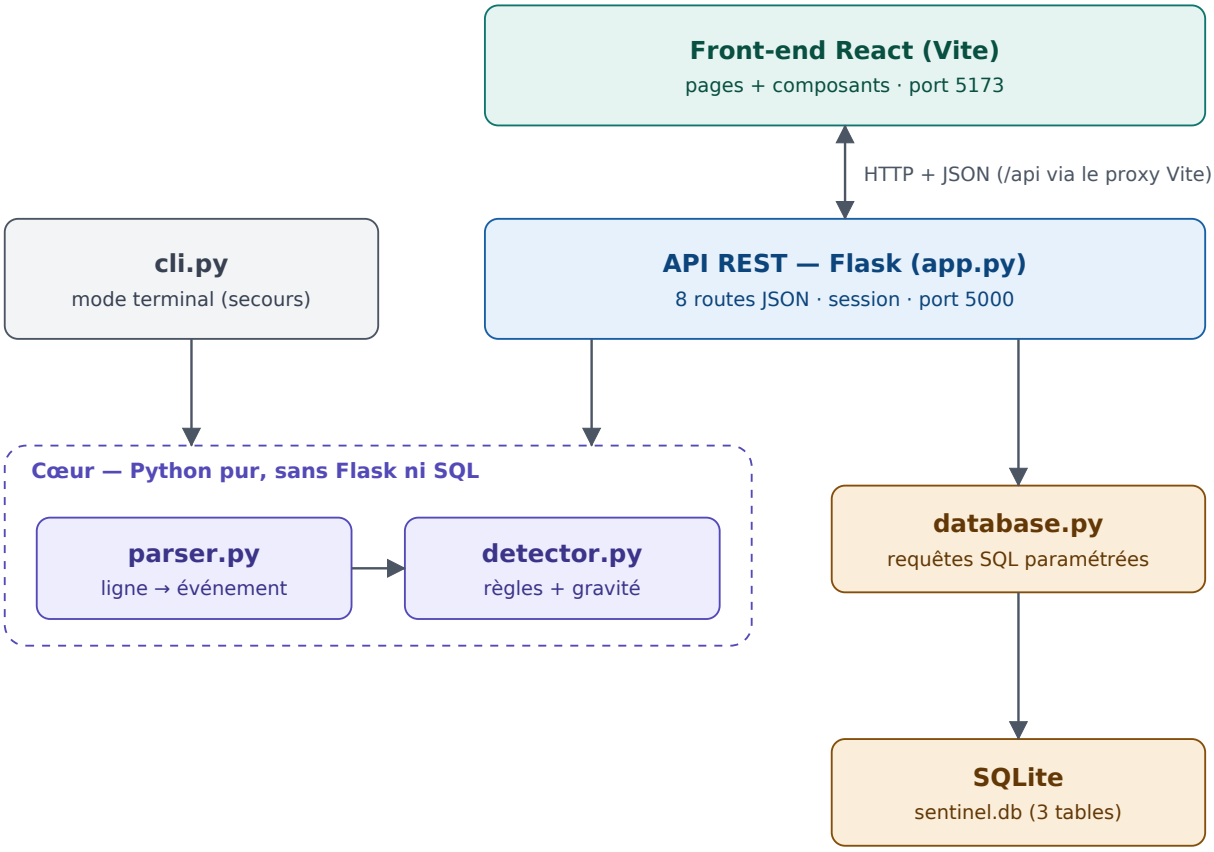
Administrateurs système et analystes sécurité juniors qui gèrent un serveur Linux (Ubuntu) exposé en SSH, dans une petite structure, sans équipe de sécurité dédiée ni SIEM.

> **Persona — Karim, 26 ans, administrateur système dans une PME de 40 personnes.** Il gère seul un serveur Ubuntu accessible en SSH. Il sait que des robots essaient de se connecter, mais il n'a ni le temps de lire `auth.log`, ni le budget pour un SIEM. Ce qu'il veut : savoir en deux minutes si quelqu'un a réussi à entrer, et quelles IP bloquer.

3.5 Type d'application

SentinelSSH est une **application web** utilisée en local, construite en 3 couches :

Couche	Technologie	Rôle
Front-end	React (avec Vite), React Router, Recharts	Interface responsive de 4 pages : connexion, accueil (envoi, historique, récidivistes), détail d'une analyse, confidentialité
API	API REST en JSON, avec Flask	Porte d'entrée du serveur : 8 routes, dont 3 pour la connexion et 5 pour envoyer, lister, consulter et supprimer des analyses
Back-end	Python (cœur d'analyse)	Lecture des logs, règles de détection, vérification des connexions
Base de données	SQLite	3 tables : compte administrateur, analyses, IP suspectes



Architecture prévue. Le cœur d'analyse ne dépend ni du web ni de la base : il est testable seul et réutilisé par un mode terminal de secours.

3.6 Pourquoi cette idée plutôt que les autres

Faisabilité

Le cœur du projet (lecture des logs et détection) est en Python, mon langage le plus solide. Le périmètre a été volontairement limité : analyse à la demande plutôt qu'en temps réel, 4 pages simples, 3 tables, 8 routes, un seul compte administrateur. La seule technologie nouvelle, React, est identifiée comme un risque et prévue dès la première semaine (§3.9).

Innovation : ce que SentinelSSH apporte par rapport à fail2ban

	fail2ban	SentinelSSH
Fonctionnement	Surveille en continu et bloque automatiquement	Analyse un fichier à la demande
Succès après échecs	Non signalé	Alerte **critique** : compromission possible
Interface	Configuration et ligne de commande	Application web : tableau, graphique, historique
Décision de blocage	Automatique	Humaine : l'outil propose, l'admin décide

SentinelSSH ne remplace pas fail2ban, il le complète : fail2ban *agit*, SentinelSSH aide à *comprendre* ce qui s'est passé.

Alignement avec mes objectifs

Le projet est défensif et centré sur l'infrastructure, ce qui correspond à ma spécialisation visée (sécurité des infrastructures et des réseaux, GRC). Il mobilise les 4 couches attendues au titre RNCP niveau 5 (front, API, back, base de données), et touche aussi à la conformité : une adresse IP est une donnée personnelle au sens du RGPD, ce qui impose de réfléchir à ce que l'on stocke et combien de temps.

Il permet enfin de démontrer les compétences du titre « Développeur web et web mobile » : maquettes, interface dynamique en React, base de données relationnelle, composants métier côté serveur, application sécurisée et déploiement documenté.

3.7 Fonctionnalités clés et objectifs SMART

Fonctionnalités clés du MVP

1. **Analyse d'un fichier de logs :** envoi d'un fichier `auth.log` (.log ou .txt, 5 Mo maximum) et lecture des tentatives de connexion.
2. **Détection et gravité :** 3 règles — attaque persistante (au moins 20 échecs, gravité moyenne), force brute rapide (au moins 5 échecs en 60 secondes, élevée), compromission possible (connexion réussie après au moins 5 échecs, critique).
3. **Visualisation :** résumé chiffré, tableau des IP suspectes trié par gravité, graphique des IP les plus actives, commande de blocage à copier.
4. **Historique en base de données :** liste des analyses passées, suppression, et repérage des IP présentes dans plusieurs analyses.
5. **Accès sécurisé :** connexion par un compte administrateur (mot de passe haché), page de confidentialité (RGPD), interface utilisable sur mobile.

Objectifs SMART

Spécifique, Mesurable, Atteignable, Réaliste, Temporel : chaque objectif a une preuve et une échéance (S1 à S4 = semaines 1 à 4 du développement).

Objectif	Mesure (la preuve)	Échéance
O1 — Détection	Les 3 règles donnent le résultat attendu sur 100 % des fichiers de test (tests automatiques au vert)	Fin S1
O2 — API et base	Les 8 routes de l'API renvoient les bonnes données et les bons codes HTTP (refus 401 sans connexion), avec les résultats enregistrés dans SQLite	Fin S2
O3 — Interface	Une fois connecté, on passe de l'envoi du fichier à l'affichage du tableau et du graphique en 3 clics maximum, sur ordinateur comme sur mobile	Fin S3
O4 — Robustesse	Les 655 146 lignes du dataset public OpenSSH de Loghub sont analysées sans plantage ; le temps d'analyse est mesuré et documenté	Fin S3
O5 — Qualité	Au moins 80 % du code d'analyse couvert par les tests ; tests lancés automatiquement à chaque envoi de code ; application déployée en local en suivant uniquement le README	Fin S4

3.8 Périmètre du projet

Dans le périmètre (in-scope)	Hors périmètre (out-of-scope)
✓ Envoi d'un fichier de logs SSH (format classique Ubuntu) ✓ Détection par 3 règles et 3 niveaux de gravité ✓ Stockage des analyses et des IP suspectes dans SQLite ✓ API REST de 8 routes en JSON ✓ Connexion par un compte administrateur unique (mot de passe haché) ✓ Interface React responsive : connexion, accueil, détail, confidentialité ✓ Tableau trié, graphique, commande de blocage à copier ✓ Historique, suppression d'une analyse, IP récidivistes ✓ Mode terminal de secours ✓ Tests automatiques et intégration continue ✓ Mise en production locale documentée (Gunicorn)	✗ Blocage automatique des IP (demanderait les droits root) ✗ Surveillance en temps réel des logs ✗ Plusieurs comptes et gestion des rôles (un seul administrateur) ✗ Lecture de journald et des autres formats de logs ✗ Autres protocoles que SSH ✗ Géolocalisation des IP ✗ Déploiement public sur Internet

Les éléments hors périmètre ne sont pas abandonnés : ils forment la feuille de route après le demo day (V2 : plusieurs comptes, journald, audit de sshd_config ; V3 : temps réel, plusieurs serveurs ; plus tard : honeypot et SIEM).

3.9 Risques et mesures d'atténuation

Risque	Probabilité	Impact	Mesure d'atténuation
Peu d'expérience avec React	Élevée	Élevé	Tutoriel officiel de React dès la semaine 1, en parallèle du back-end · API terminée et testée avant le front · interface limitée à 4 pages simples
Front et back qui n'arrivent pas à communiquer (CORS)	Moyenne	Élevé	Proxy de développement Vite · API testée seule avec curl avant de brancher React
Formats de logs différents selon les systèmes	Moyenne	Moyen	Un seul format ciblé (Ubuntu classique) · lignes non reconnues comptées et affichées
Seuils de détection mal calibrés	Moyenne	Moyen	Seuils réglables dans un fichier de configuration, validés sur des fichiers de test
Failles dans l'outil lui-même (injection SQL, XSS)	Faible	Élevé	Requêtes SQL paramétrées · affichage échappé par React · taille et type de fichier contrôlés
Connexion mal sécurisée	Faible	Élevé	Fonctions éprouvées de Werkzeug (hachage) et sessions Flask · aucun mécanisme « fait maison » · tests dédiés
Projet solo : pas de relecture	Élevée	Moyen	Tests automatiques, Pull Requests relues à froid, tests par des étudiants de la promo
Retard sur le planning	Moyenne	Élevé	Construction par couches (cœur, puis API, puis front) · ordre de coupe défini : bonus, puis graphique, puis récidivistes

3.10 Défis et opportunités identifiés pendant le brainstorming

Défis	Opportunités
• Apprendre React en même temps que le reste du projet. • Tenir un projet complet (4 couches) seul en 1 mois. • Calibrer des règles de détection ni trop sensibles, ni trop laxistes. • Rendre sûr un outil qui affiche des données écrites par des attaquants.	• Un projet de portfolio directement lié à la cybersécurité défensive. • Un angle conformité (RGPD) cohérent avec mon objectif GRC. • Des données de test publiques (Loghub) pour prouver la robustesse. • Une base pour les versions suivantes : audit de configuration, temps réel, honeypot.

3.11 Prochaines étapes

Étape	Contenu
Stage 2 — Planification	Charte de projet, planning détaillé, découpage en issues GitHub
Semaine 1	Cœur d'analyse en Python (lecture des logs, 3 règles) + tests · tutoriel React en parallèle
Semaine 2	Base de données SQLite + API REST Flask + connexion + tests de l'API + intégration continue
Semaine 3	Interface React responsive : connexion, accueil, détail, confidentialité, tableau, graphique
Semaine 4	Gel des fonctionnalités : mise en production locale, corrections, documentation, répétitions du demo day

4. Résumé du MVP sélectionné

Le problème. Un serveur Linux exposé en SSH reçoit en permanence des tentatives de connexion malveillantes, enregistrées dans des fichiers de logs volumineux. Les analyser à la main est long et peu fiable : une attaque réussie peut passer inaperçue au milieu de milliers de lignes.

La solution. SentinelSSH est une application web d'analyse de logs SSH. L'administrateur envoie son fichier `auth.log` ; un back-end Python applique 3 règles de détection (attaque persistante, force brute rapide, compromission possible) et attribue une gravité à chaque IP suspecte. Les résultats sont enregistrés dans une base SQLite et présentés par une interface React, accessible après connexion : résumé, tableau trié par gravité, graphique, commande de blocage à copier et historique des analyses.

Pourquoi ce choix. Parmi les 6 idées explorées, SentinelSSH obtient le meilleur score (26/30). Il est défensif et centré sur l'infrastructure, donc aligné avec ma spécialisation en sécurité des réseaux et GRC. Il est complet techniquement, avec des raisons réelles d'avoir un front, une API, un back et une base de données. Enfin, il se distingue de fail2ban : il signale la compromission possible, explique au lieu de bloquer à l'aveugle et laisse la décision à l'humain. Son périmètre a été réduit pour tenir en 1 mois.

L'impact potentiel. SentinelSSH donne à un administrateur junior un premier niveau de surveillance simple, sans déployer d'outil lourd : en quelques secondes, il sait quelles IP l'attaquent et si l'une d'elles a réussi à entrer. Le projet respecte la minimisation des données (seules les IP suspectes sont stockées, jamais le log complet) et pose les bases de versions futures : audit de configuration, surveillance en temps réel, puis honeypot et intégration à un SIEM.

> **En une phrase :** SentinelSSH transforme un fichier de logs illisible en une liste claire de menaces classées par gravité, pour qu'un administrateur sache en quelques secondes qui attaque son serveur et si quelqu'un a réussi à entrer.

Annexe — Journal des décisions

Chaque décision importante du Stage 1 est tracée ici. Le journal continue pendant les stages suivants.

ID	Date	Décision	Options envisagées	Raison
D1	29/09/2026	Idée retenue : SentinelSSH	Les 6 idées du §2.4	Meilleur score (26/30) ; défensif, complet, différent de fail2ban
D2	29/09/2026	Analyse à la demande	Temps réel	Faisable en 1 mois ; le temps réel passe en V3
D3	29/09/2026	Front-end en React (Vite)	HTML/CSS simple, Vue	Une technologie JavaScript est exigée côté front ; React est le standard le plus répandu
D4	29/09/2026	Back-end en Python/Flask	Node.js/Express, FastAPI, Django	Le cœur de détection reste dans mon langage le plus solide ; Flask est le plus léger
D5	29/09/2026	Base de données SQLite	PostgreSQL, MongoDB	Usage local, un seul utilisateur, données reliées : pas besoin d'un serveur de base

ID	Date	Décision	Options envisagées	Raison
D6	29/09/2026	Blocage suggéré, pas automatique	Blocage automatique par iptables	Éviter de donner les droits root à l'application ; décision humaine ; différence avec fail2ban
D7	30/09/2026	Connexion par un compte administrateur unique	Aucune connexion (usage local), plusieurs comptes	Application sécurisée attendue par le titre RNCP ; un outil de sécurité doit protéger ses propres données ; un seul compte garde le projet simple
D8	30/09/2026	Mise en production locale avec Gunicorn	Serveurs de développement, Docker	Compétence « documenter le déploiement » ; une seule commande pour la démo ; Docker ajouterait un outil à apprendre